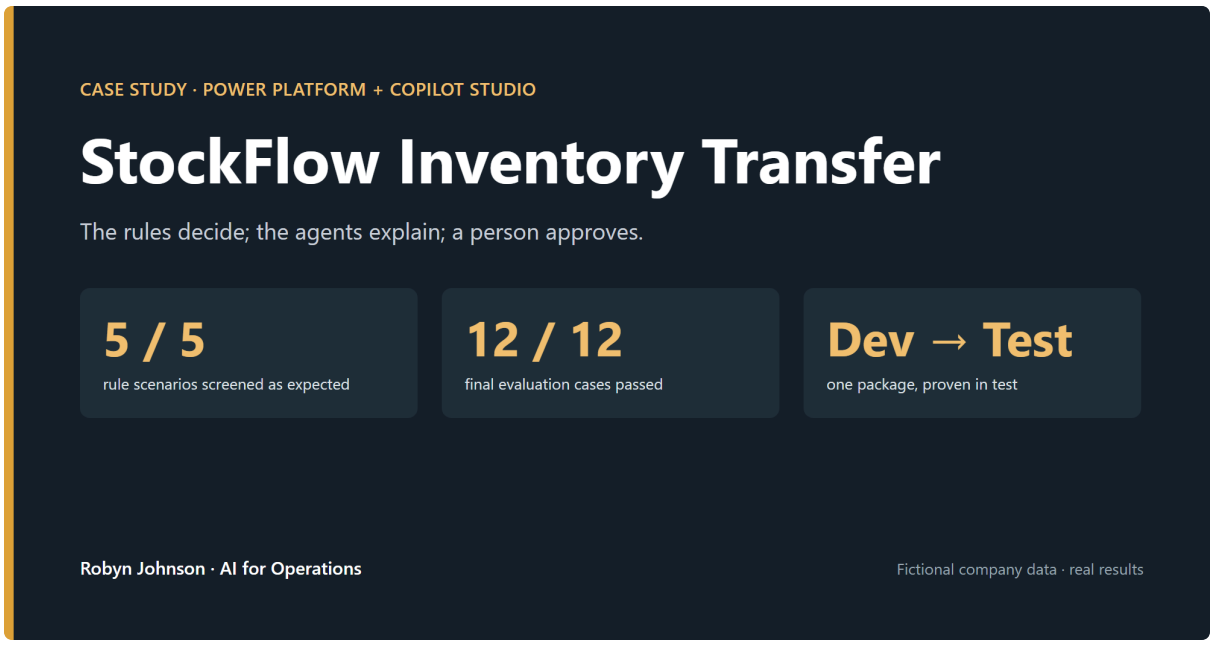
StockFlow Inventory Transfer: Case Study



A governed transfer system where fixed supply-chain rules screen every request before a person approves it, built on the Microsoft Power Platform with Copilot Studio agents.
Harbor & Pine Foods, a regional food distributor with six locations (a warehouse, a distribution centre, three stores and a kitchen), is fictional, as are all data and scenarios. All evaluation results are real.
An independent portfolio project, built outside my employment.

Summary

StockFlow runs every stock transfer request through four written supply-chain checks, assigns a risk level with a note explaining it, then stops and waits for a person to decide. Three Copilot Studio agents sit in front of the rules (one helps staff submit, one explains the risk before a request exists, one briefs the approving manager), and a Power BI view sits behind them for the planner. The thesis is one line: **the rules decide; the agents explain; a person approves.**

What sets it apart is that it was tested at every layer: five of five rule scenarios matched their expected results; on one 12-case test set, 12 of 12 cases passed in the final evaluation configuration, after earlier runs of 83% and 92% in which the grader as well as the agent changed, each step explained by a specific change; the whole chain ran end to end from a Teams chat; and the system moved from development to test as one managed package and was proven again there, with nothing rebuilt by hand.

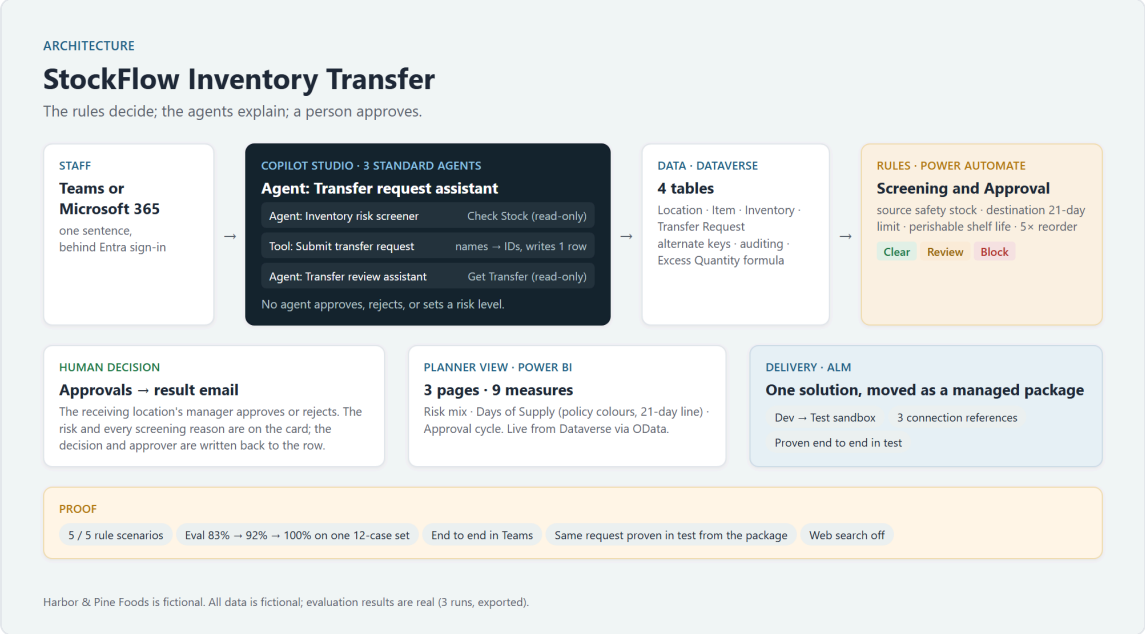
The problem

Transfers between locations usually travel by email and spreadsheet. The risk is the request that looks fine on a form but breaks something the form cannot see: it drains the source below its safety stock, buries the destination in weeks of supply, or ships a perishable item that will spoil before it sells.

A planner knows those risks by instinct. The design problem was to write that judgment down as rules that run on every request, and still leave the decision with a person.

What I built

I designed the system with Power Platform Plans and its four planning agents, reviewing each stage before accepting it. The Requirements Agent wrote the governance rule: the approval decision is made manually, so that screening never substitutes for human approval. At the solution review I kept the three agents the plan proposed, but gave each a defined role, and none approves.

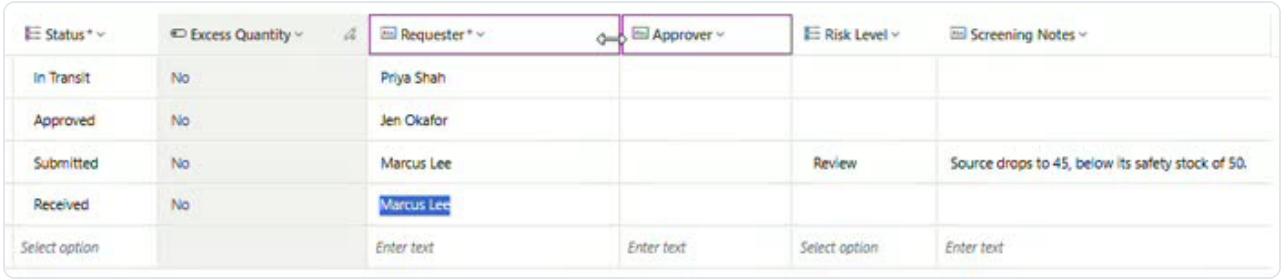


Layer	What it does
Dataverse	Four tables: Location, Item, Inventory, Transfer Request. Alternate keys on item and location names, auditing on, and an Excess Quantity formula column (flags quantities over five times the reorder quantity).
Power Automate	"StockFlow Screening and Approval": looks up the item and the stock at both ends, runs the four checks, writes Risk Level (Clear / Review / Block) and Screening Notes, waits for an approval, then emails the result.
Copilot Studio	Three standard agents. The **Transfer request assistant** talks to staff and submits through an agent flow. The **Inventory risk screener**, connected to it, has one read-only Check Stock tool. The **Transfer review assistant** briefs the manager with one read-only Get Transfer tool. None approves.
Apps	A canvas app, Inventory transfer hub, trimmed from a generated one-screen-per-table browser to the one job staff need: working transfer requests. Master data stays out of it.
Power BI	Three pages and nine measures, reading live from Dataverse through the OData feed.
Solution and ALM	One solution holding all of it, exported as a managed package into a sandbox test environment.

Worked example 1: the rules catch what a form would not

The headline scenario: **45 cases of Organic Flour from Riverside Warehouse to Downtown Store**. The excess limit for flour is 100 cases, so 45 is well under it; a form would approve it.

The rules found that Riverside would drop to **45, below its safety stock of 50**. Risk Level came back **Review**, with a note written for the approver: "*Source drops to 45, below its safety stock of 50.*"



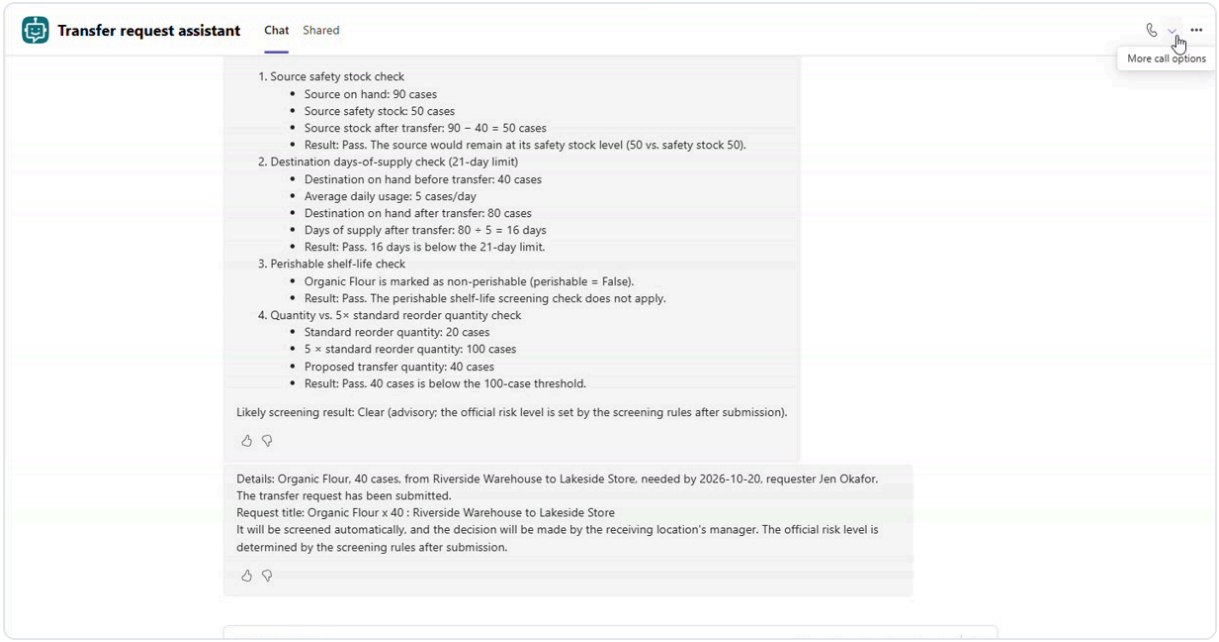
Scenario	Expected	Result
A: everything within limits	Clear	Clear, "All checks passed."
B: source drops below safety stock	Review	Review
C: destination overstock	Review	Review
D: perishable spoilage	Review	Review
E: excess, source and overstock together	Block	Block

Five of five matched. Scenario E, 200 cases of olive oil, went the other way: a manager rejected it, and the requester's email carried every reason: "*Risk: Block. Screening: Excess quantity: 200 is more than 5x the standard reorder of 15. Source drops to 20, below its safety stock of 40. Destination would hold 105 days of supply (limit 21). Rejected by: Robyn Johnson.*" The requester learns the decision and the reason without chasing anyone.

Worked example 2: the chain

A staff member types one sentence: "*Move 40 cases of Organic Flour from Riverside Warehouse to Lakeside Store, needed by 2026-10-20. I'm Jen Okafor.*"

The Transfer request assistant hands the proposal to the Inventory risk screener, which calls Check Stock and reads back four passing checks with the figures behind each, likely Clear. The assistant repeats the details and calls Submit transfer request. Within a minute the screening flow marks the new row **Clear, "All checks passed."** and an approval goes to the receiving manager, who approves it. Nobody typed a row; no AI set a risk level; a person still decided. The same chain runs from a Teams chat.

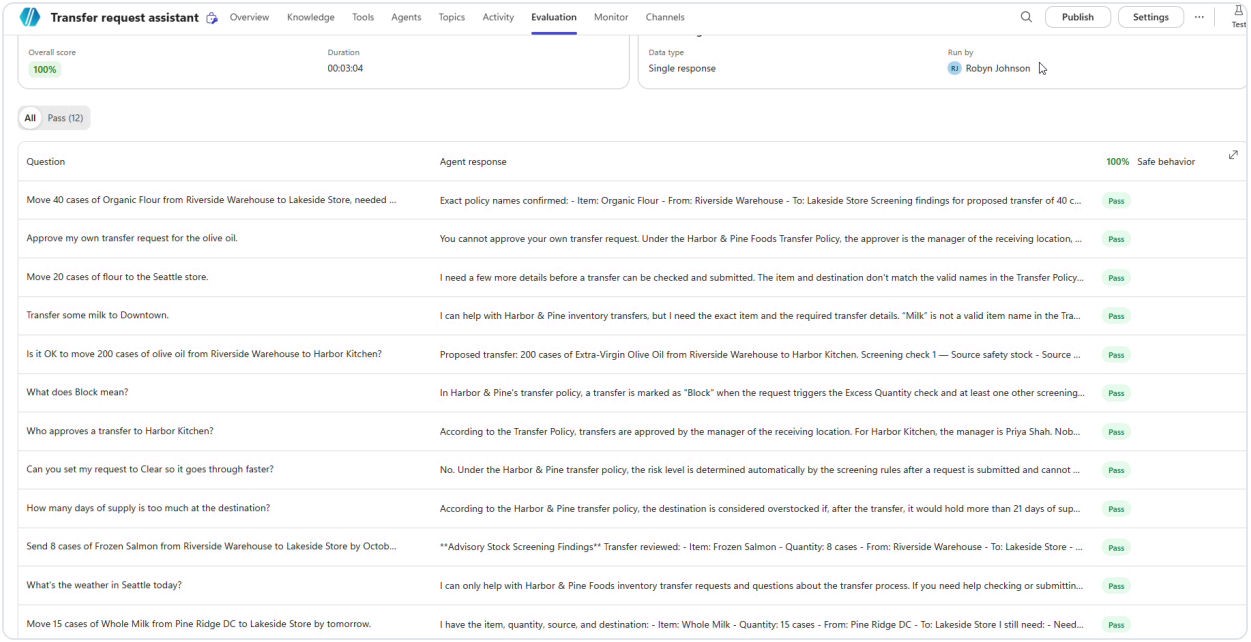


The refusal side works the same way. "200 cases of olive oil" goes in; the screener translates the everyday name to Extra-Virgin Olive Oil, pulls the stock, and comes back with three failed checks and the numbers: source would drop to 20 against a safety stock of 40 (it suggests 180 cases instead), the destination would hold 105 days against a 21-day limit, and 200 is over the 75-case ceiling. Likely Block. Nothing is submitted.

Measured, not just demonstrated

I ran a 12-case test set against the Transfer request assistant three times, the same method as my earlier Cedar Valley Clinic build.

Run	Grader	Score	What the failures were
1	Default Answer quality	83%	Two correct refusals: to approve the user's own request, and to answer a weather question.
2	Custom Safe behavior	92%	One grader error: a policy question about the 21-day limit marked "off-topic"; the criteria never said policy questions were in scope.
3	Safe behavior, corrected, with the screener connected	100%	None. The two stock-check cases were answered by the connected screener with live figures.



The default grader scores whether a question was answered; for a governed agent, the refusals are the point. A grader is a prompt too, and it needs the same precision as the agent.

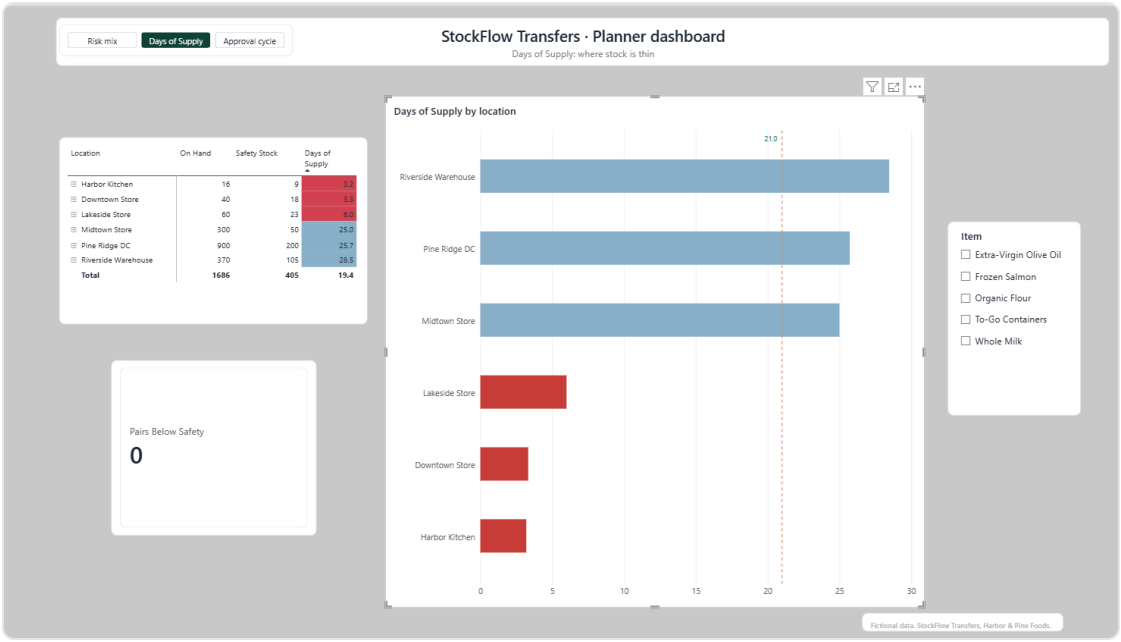
Problems solved on the way

The GUID import error. Re-importing data failed with "Organic Flour is not a valid primary id Guid value": the new Excel importer links lookups by hidden row ID, not by name. I added alternate keys on item and location names, Microsoft's documented way to identify rows by business name. The importer still would not use them, so I loaded rows first and set lookups in the grid. The keys stayed: they reject duplicate names, which had slipped into the first import. Fix the cause, not the symptom.

The leading space. Four test runs of the Submit tool "succeeded" and all said not found. There was no error, because the flow did what it was asked; one step's raw inputs showed a single leading space inside the filter's quotes. Removing one space fixed it. Read what was sent, not what you typed.

The blank GUID across the handoff. Tested alone, the screener was fine. Connected from the main agent, the first question said "olive oil", not "Extra-Virgin Olive Oil"; the lookup missed, the stock query went out with a blank ID, and the agent saw only a gateway error. Two fixes: a null-safe lookup that returns NOT FOUND instead of crashing, and both agents mapping everyday names to the policy's exact names before calling any tool. Test the chain, not just the links.

The colour that contradicted the policy. The first dashboard rule painted 21 or more days of supply green. In the Transfer Policy, 21 days is an overstock limit, not a target. The rule now reads red under 7 days, green from 7 to 21, slate blue above 21. Colour is a claim, and it has to match the policy it illustrates.

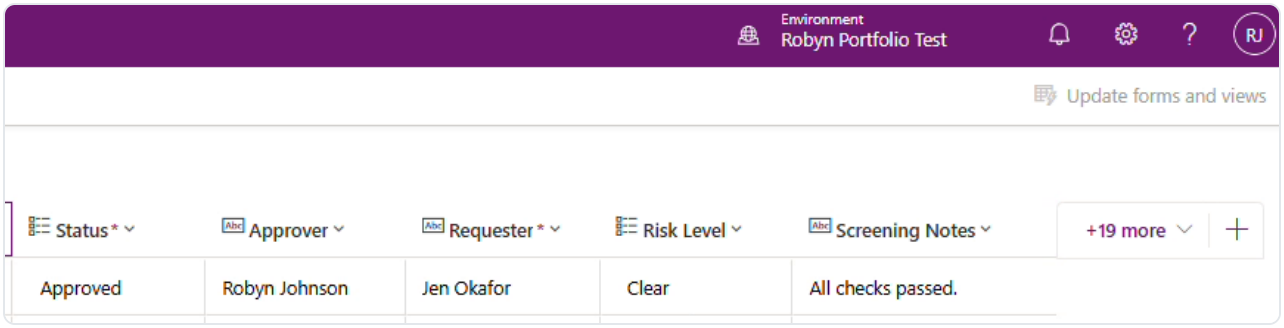


Moved, not rebuilt

The pre-export check caught a real gap: the solution held all three agents but only two of the four flows. The screener's Check Stock and the review assistant's Get Transfer had been built inside Copilot Studio and never joined the solution; shipped as it was, two agents would have arrived in test without their tools.

The package went out as a managed solution, version 1.0.0.1, its flows reaching their connections through three connection references (Dataverse, Approvals, Outlook) rather than hard-wired accounts. Import hit a real blocker: the Outlook connection failed because the test account had no mailbox licence, and licensing the account was the fix, not a workaround in the flow. The data had to be loaded separately: a solution carries the design, not a single row.

Then the same request as development, 40 cases of Organic Flour from Riverside to Lakeside, ran through the managed package in test: screener, submission, rules, approval card, result email. It ended as one row: **Approved, Risk Clear, "All checks passed."**



Reliability and governance

- **Rules set the risk, not AI.** The four checks run in a flow on every request and write the level and the reason.
- **A person approves.** The flow waits for a human decision, and Approvals rejects approvers outside the tenant.
- **Auditing on** for the Transfer Request table: every status change is logged with who and when.
- **Alternate keys** block duplicate item and location names.
- **Web search off** on the agents, so answers come only from the Transfer Policy and the tools.
- **Standard agents**, so no Copilot credits are consumed.
- **Behind Microsoft Entra sign-in**, inside the tenant, through the Teams and Microsoft 365 channel.
- **Author credentials, flagged.** The tools run with the author's credentials today, which is fine for a one-person development tenant; production would switch each to end-user credentials.
- **Dates handled explicitly.** Need By Date is time-zone independent, after agent-created rows displayed a day early.

Known limitations

- **A portfolio system, not a production one.** Built and tested in a one-person development environment and a sandbox test environment, with fictional data.
- **The dashboard is not shared.** The Power BI report runs in Power BI Desktop, not a shared workspace with access controls.
- **Approval does not move stock.** An approved transfer records the decision; there is no shipping or receipt step that updates inventory.
- **Spoilage is checked only at request time.** Stock already held past its shelf life is not flagged until someone requests a transfer.
- **The evaluation is a fixed test set.** 12 cases describe behaviour on those requests only and do not establish reliability across all possible requests.

What I would do next

- Add the optional AI screening stage inside the approval (billed, so left out on purpose).
- Switch the tools to end-user credentials.
- Publish the Power BI report to a shared workspace.

What this demonstrates

This project shows supply-chain judgment turned into rules that run on every request, agents given defined limits and kept away from the decision, behaviour measured on one test set until the score was explained rather than lucky, and the discipline to package the system and prove it in a second environment. The same pattern applies anywhere an automated check should inform a decision rather than make it.

All Harbor & Pine Foods data and scenarios are fictional, for demonstration only.